

Pontifícia Universidade Católica de Minas Gerais

Algoritmos em Grafos - 2026/1

Relatório Técnico

SORL: Sistema de Otimização de Rotas Logísticas

Trabalho Prático 01/2026

Matheus Filipe de Deus Silva e Thiago Villani Marinho

Instituição	Pontifícia Universidade Católica de Minas Gerais
Disciplina	Algoritmos em Grafos
Trabalho	Trabalho Prático 01/2026
Linguagem	C# (.NET 8.0)

1. Modelagem da Rede Logística

A rede de transporte da Entrega Máxima Logística S.A. foi modelada como um grafo direcionado e ponderado $G = (V, E)$, onde cada elemento representa um componente real da malha logística:

- Cada vértice $v \in V$ representa um hub logístico (centro de distribuição, porto ou polo industrial).
- Cada aresta $(u, v) \in E$ representa uma rota rodoviária direta do hub u para o hub v .
- Cada aresta carrega dois atributos numéricos: peso (custo financeiro em R\$ por unidade de carga) e capacidade (limite diário em toneladas da rota).

Os grafos foram fornecidos no formato DIMACS estendido: a primeira linha declara V (número de vértices) e E (número de arestas); as demais linhas descrevem cada aresta com quatro valores - origem, destino, peso e capacidade - com vértices numerados a partir de 1. Internamente, o sistema converte os índices para base 0 durante o processamento.

Essa modelagem captura fielmente os dois aspectos econômicos relevantes de cada rota: o custo por envio (atributo peso, utilizado em P1, P3, P4 e P5) e a restrição de volume diário (atributo capacidade, utilizado exclusivamente em P2). A separação dos atributos permite que cada problema consulte apenas as informações que lhe são pertinentes, sem ambiguidade.

2. Estrutura de Dados

2.1 Seleção automática por densidade

O sistema calcula automaticamente a densidade do grafo antes de instanciar a estrutura de dados. A fórmula utilizada é:

$$\text{densidade} = E / (V \times (V - 1))$$

O denominador $V \times (V - 1)$ representa o número máximo de arestas possíveis em um dígrafo sem laços próprios. O limiar adotado é 0,30: grafos com densidade $\geq 0,30$ utilizam Matriz de Adjacência; os demais utilizam Lista de Adjacência.

A tabela abaixo apresenta os resultados da seleção automática para os sete grafos do trabalho:

Grafo	V	E	Densidade	Estrutura Escolhida
grafo01	6	12	0,400	Matriz de Adjacência
grafo02	5	6	0,300	Matriz de Adjacência
grafo03	8	10	0,179	Lista de Adjacência
grafo04	10	15	0,167	Lista de Adjacência

Grafo	V	E	Densidade	Estrutura Escolhida
grafo05	10	30	0,333	Matriz de Adjacência
grafo06	50	200	0,082	Lista de Adjacência
grafo07	100	400	0,040	Lista de Adjacência

O limiar 0,30 foi escolhido deliberadamente para que ambas as estruturas apareçam entre os grafos fornecidos, demonstrando o mecanismo de seleção automática operando em ambas as direções. grafo01, grafo02 e grafo05 utilizam Matriz; grafo03, grafo04, grafo06 e grafo07 utilizam Lista.

2.2 Justificativa técnica de cada estrutura

Lista de Adjacência

Preferível em grafos esparsos (densidade $< 0,30$), como grafo03 a grafo07, pelos seguintes motivos:

- Consumo de memória $O(V + E)$, proporcional apenas às arestas existentes e não ao quadrado de vértices.
- O método Vizinhos(v) retorna em tempo $O(\text{grau}(v))$ apenas os vizinhos reais, sem varrer posições vazias.
- Inserção de arestas em $O(1)$ no final da lista interna.

Matriz de Adjacência

Preferível em grafos densos (densidade $\geq 0,30$), como grafo01, grafo02 e grafo05, pelos seguintes motivos:

- Acesso direto existe(u, v) em $O(1)$, benéfico em algoritmos que testam a existência de arestas específicas repetidamente.
- Para $V \leq 100$ e densidade alta, o custo de memória $O(V^2)$ é aceitável e oferece acesso mais rápido.
- Três matrizes internas (peso, capacidade, existe) permitem distinguir ausência de aresta de aresta com peso zero.

Ambas as representações implementam a interface `IRepresentacao`, garantindo que todos os algoritmos funcionem corretamente e de forma idêntica, independentemente da estrutura interna escolhida para aquele grafo específico.

3. Resolução dos Problemas

3.1 Problema I - Roteamento de Menor Custo

Pergunta de negócio: Qual é o trajeto mais econômico para enviar cargas entre dois hubs?

Algoritmo utilizado: Dijkstra - $O(V + E)$

Justificativa:

O problema pede o caminho de menor custo total (soma dos pesos das arestas) entre dois hubs em um grafo direcionado. O algoritmo de Dijkstra resolve o problema de caminho mínimo de fonte única com complexidade $O(V + E)$ ao usar uma fila, sendo correto e eficiente quando todos os pesos são não-negativos. Essa condição é garantida pelos dados do problema, pois custos financeiros de transporte não podem ser negativos.

O algoritmo de Bellman-Ford seria uma alternativa válida, porém tem complexidade $O(V \times E)$, superior à do Dijkstra para grafos sem pesos negativos. Floyd-Warshall resolveria todos os pares de vértices, mas é desnecessário quando a consulta é entre dois vértices específicos.

Modelagem: Grafo direcionado. O custo de transporte de A para B pode diferir do custo de B para A (diferentes condições de tráfego, pedágio, sentido de via).

Implicação logística: O caminho de menor custo indica a sequência de hubs pela qual a carga deve ser roteada para minimizar o gasto total de transporte. A empresa pode adotar esse roteiro como política padrão de despacho entre os centros analisados.

3.2 Problema II - Capacidade Máxima de Escoamento

Pergunta de negócio: Qual é o volume máximo diário de mercadorias que a rede consegue escoar entre dois hubs?

Algoritmo utilizado: Edmonds-Karp - $O(V \times E^2)$ + corte mínimo pelo Teorema do Fluxo Máximo e Corte Mínimo.

Justificativa:

O problema modela a rede logística como uma rede de fluxo, onde cada aresta possui uma capacidade máxima diária. O algoritmo de Edmonds-Karp é uma implementação do Ford-Fulkerson que utiliza BFS para encontrar sempre o caminho aumentante mais curto. Isso garante terminação em tempo polinomial $O(V \times E^2)$, diferentemente do Ford-Fulkerson, que pode não terminar para capacidades maiores.

A escolha pelo Edmonds-Karp em detrimento de algoritmos mais avançados, como Dinic com complexidade $O(V^2 \times E)$, foi motivada pela simplicidade de implementação e pela adequação às dimensões dos grafos fornecidos ($V \leq 100$). Para redes de grande escala, Dinic seria preferível.

Após atingir o fluxo máximo, uma BFS no grafo residual identifica o conjunto de vértices alcançáveis a partir de S. As arestas originais que cruzam desse conjunto para o lado não-alcançável formam o corte mínimo, resultado direto do Teorema do Fluxo Máximo e Corte Mínimo.

Modelagem: Grafo direcionado, pois fluxo de carga tem sentido definido (origem $\rightarrow$ destino). O grafo residual inclui arestas reversas para permitir cancelamento de fluxo enviado em iterações anteriores.

Implicação logística: O fluxo máximo indica o volume máximo diário (em toneladas) que a rede consegue escoar do hub central até o terminal de destino. As arestas do corte mínimo são os gargalos estruturais da malha, rotas prioritárias para investimento em ampliação de capacidade.

3.3 Problema III - Expansão da Rede de Comunicação (AGM)

Pergunta de negócio: Como interligar todos os hubs com o menor custo de instalação de fibra óptica?

Algoritmo utilizado: Kruskal - $O(E \log E)$

Justificativa:

O problema pede conectar todos os hubs com custo mínimo sem formar ciclos, definição exata de Árvore Geradora Mínima (AGM). O algoritmo de Kruskal ordena as arestas por peso crescente e inclui cada aresta na árvore se ela não formar ciclo com as já incluídas, atingindo complexidade $O(E \log E)$.

O algoritmo de Prim com fila de prioridade seria equivalente em complexidade assintótica; Kruskal foi escolhido por ser mais direto para grafos esparsos, que representam a maioria dos grafos fornecidos (5 de 7 utilizam Lista de Adjacência).

Decisão de modelagem por grafo não-direcionado: Fibra óptica é um recurso físico bidirecional: um cabo instalado entre A e B serve tanto para $A \rightarrow B$ quanto para $B \rightarrow A$. Por isso, a AGM foi calculada tratando o grafo como não-direcionado, normalizando cada par (u, v) para evitar duplicação de arestas simétricas.

Implicação logística: As arestas da AGM indicam quais rotas físicas devem receber o cabeamento de fibra óptica, garantindo que todos os hubs fiquem interligados com o menor investimento total possível, sem redundâncias de infraestrutura.

3.4 Problema IV - Agendamento de Manutenções sem Conflito

Pergunta de negócio: Como alocar rotas em turnos de manutenção sem que duas rotas conflitantes sejam mantidas simultaneamente?

Algoritmo utilizado: Welsh-Powell

Justificativa da modelagem:

O problema define que duas rotas conflitam quando compartilham um hub (pátio de manutenção, oficina ou equipamento). Essa relação de conflito é capturada pelo grafo linha: cada aresta do grafo original torna-se um nó do grafo linha, e dois nós são adjacentes se as rotas originais compartilham pelo menos um vértice. Colorir os nós desse grafo linha, atribuindo uma cor (turno de manutenção) a cada rota, equivale a uma coloração de arestas do grafo original.

Justificativa do algoritmo:

O algoritmo Welsh-Powell ordena os nós do grafo linha por grau decrescente e aplica coloração gulosa: para cada nó, atribui a menor cor não utilizada por nenhum vizinho já colorido. Esse procedimento produz uma solução válida (sem conflitos) com complexidade $O(m^2)$ para m arestas.

Limitação declarada: A coloração mínima (número cromático) é um problema NP-difícil. Welsh-Powell é uma heurística gulosa que fornece um limitante superior para o número de turnos necessários, não necessariamente o mínimo absoluto. Esse comportamento é inerente à classe de algoritmos gulosos para coloração e é aceitável para o contexto do problema.

Implicação logística: O número de turnos indica o tempo mínimo estimado (em turnos de trabalho) para concluir todas as manutenções. A alocação rota $\rightarrow$ turno define o cronograma operacional: todas as rotas de um mesmo turno podem ser mantidas simultaneamente sem conflito de recursos compartilhados.

3.5 Problema V - Rota Única de Inspeção

Pergunta de negócio: *É possível criar um percurso único de inspeção que cubra todas as rotas ou todos os hubs sem repetição?*

Cenário A - Percurso de Rotas (Circuito Euleriano)

Algoritmo utilizado: Verificação de graus + DFS dupla (conectividade forte) + DFS que consome arestas

Justificativa:

Um circuito euleriano percorre todas as arestas exatamente uma vez e retorna ao ponto de partida. Para um dígrafo, as condições necessárias e suficientes são: (1) todo vértice com arestas tem grau de entrada igual ao grau de saída; e (2) o grafo é fortemente conexo, considerando apenas os vértices com arestas.

A condição 1 é verificada em $O(V)$ com contadores de grau. A condição 2 é verificada com duas buscas em profundidade (DFS): uma no grafo original e outra no grafo transposto (arestas invertidas), partindo do mesmo vértice inicial. Se ambas alcançam todos os vértices com aresta, o grafo é fortemente conexo. Esse método usa exclusivamente DFS, algoritmo estudado em aula.

A construção do circuito utiliza uma DFS iterativa que consome arestas: ao visitar um vértice, remove uma aresta de saída disponível e avança; quando um vértice não tem mais saídas disponíveis, é inserido na lista do circuito. A ordem reversa dos vértices inseridos forma o circuito euleriano.

Modelagem: Grafo direcionado, pois o inspetor percorre as rotas no sentido do tráfego definido.

Implicação logística: Se o circuito euleriano existir, o inspetor consegue verificar todas as estradas sem repetir nenhuma, retornando ao ponto de partida - roteiro ideal de inspeção. Se não existir, o sistema informa qual condição foi violada, indicando por que a inspeção completa em circuito único não é viável na malha atual.

Cenário B - Percurso de Hubs (Ciclo Hamiltoniano)

Algoritmo utilizado: Busca em profundidade exaustiva (DFS) com retrocesso e limite de tempo de 10 segundos

Justificativa:

O ciclo hamiltoniano, visitar todos os vértices exatamente uma vez e retornar ao inicial, é um problema NP-difícil; não existe algoritmo polinomial conhecido. A abordagem adotada é a busca em profundidade exaustiva: a DFS escolhe um vértice vizinho não visitado, avança, e quando não pode continuar, retrocede pela pilha de chamadas, desfazendo a escolha e tentando o próximo vizinho. Esse retrocesso é o comportamento padrão da DFS recursiva, não um algoritmo adicional.

Dado que a DFS exaustiva pode não terminar em tempo hábil para grafos grandes (grafo06 com 50 vértices e grafo07 com 100 vértices), foi implementado um limite de tempo de 10 segundos por grafo. Ao estourar o limite, o resultado é registrado como não conclusivo dentro do limite de tempo.

Decisão de modelagem por grafo não-direcionado: O inspetor deve visitar todos os hubs, podendo usar as vias nos dois sentidos. Tratar o grafo como não-direcionado amplia as possibilidades de roteiro e é a interpretação prática mais adequada para inspeção de instalações físicas.

Implicação logística: Se o ciclo hamiltoniano existir, o inspetor pode visitar todos os hubs em uma única saída sem revisitar nenhum - roteiro ideal de vistoria. Para grafos grandes, o resultado "não conclusivo" indica limitação computacional inerente ao problema NP-difícil.

4. Decisões de Direcionamento

Cada problema exigiu uma decisão explícita sobre tratar o grafo como direcionado ou não-direcionado. A tabela abaixo resume as decisões e suas justificativas:

Problema	Tratamento	Justificativa
P1 - Menor Custo	Direcionado	Rotas têm sentido e custo específico por direção
P2 - Fluxo Máximo	Direcionado	Fluxo de carga tem origem e destino definidos
P3 - AGM (Kruskal)	Não-direcionado	Fibra óptica é bidirecional por natureza física
P4 - Coloração	Não-direcionado	Conflito de recursos é simétrico entre rotas
P5A - Euleriano	Direcionado	Inspeção respeita o sentido do tráfego nas vias
P5B - Hamiltoniano	Não-direcionado	Inspeção de hubs permite uso das vias em ambos os sentidos

5. Log de Execução

O sistema foi executado para os sete grafos fornecidos, gerando automaticamente um arquivo de log completo na pasta Logs/ do diretório de execução. O arquivo é nomeado com data e hora (ex.: resultados_20260615_143022.txt) para rastreabilidade.

Cada entrada do log contém as seguintes informações:

- Nome do arquivo do grafo e caminho completo no sistema
- Número de vértices (V) e arestas (E) declarados e efetivamente lidos
- Densidade calculada e estrutura de dados escolhida com justificativa
- Número de arestas duplicadas descartadas durante a leitura do arquivo
- [P1] Caminho de menor custo e custo total entre os vértices S e T
- [P2] Fluxo máximo e arestas do corte mínimo entre os vértices S e T
- [P3] Custo total e arestas da Árvore Geradora Mínima pelo algoritmo de Kruskal
- [P4] Número de turnos e alocação rota → turno pela coloração Welsh-Powell
- [P5A] Resultado euleriano: existe ou não existe, com motivo e sequência de vértices
- [P5B] Resultado hamiltoniano: existe, não existe ou não conclusivo por timeout
- Tempo de execução de cada algoritmo medido individualmente com Stopwatch

O log é gravado simultaneamente no console e no arquivo em disco, garantindo visibilidade durante a execução e persistência dos resultados para análise posterior.